Kwalifikacja INF.04

Zestaw zadań ćwiczeniowych

Aplikacje desktopowe (C# / WPF)

---

12 zadań WPF • wersja dla ucznia

Klasa 5 technikum – przygotowanie do egzaminu zawodowego

Rok szkolny 2026/2027

Spis treści

1. Wymagania wspólne dla wszystkich zadań 3
1.1. Aplikacje desktopowe (WPF) 3
1.2. Dokumentacja 3
2. Przegląd zadań 4
3. Zadania – aplikacje desktopowe (WPF) 5
Zadanie W1. Rejestracja konta 5
Zadanie W2. Generator haseł i dane pracownika 6
Zadanie W3. Kalkulator BMI 7
Zadanie W4. Szyfr Cezara 8
Zadanie W5. Mieszalnik kolorów RGB 9
Zadanie W6. Rezerwacja stolika 10
Zadanie W7. Przelicznik walut z historią 11
Zadanie W8. Lista zakupów 12
Zadanie W9. Quiz wiedzy informatycznej 13
Zadanie W10. Zamówienie pizzy 14
Zadanie W11. Stoper z międzyczasami 15
Zadanie W12. Weryfikacja numeru PESEL 16

Aby odświeżyć spis treści w programie Word: kliknij prawym przyciskiem myszy na spis i wybierz „Aktualizuj pole”.

Jak pracować z zestawem

Zestaw zawiera 12 zadań przygotowujących do części egzaminu INF.04 dotyczącej aplikacji desktopowych. Zadania są napisane tak jak w arkuszu egzaminacyjnym: czytaj treść dokładnie i realizuj każdy wymóg – na egzaminie oceniany jest każdy z nich osobno.

- ★☆☆ zadanie podstawowe, ★★☆ średnie, ★★★ rozszerzone.
- W miejsce <numer zdającego> wpisz swój numer z dziennika (lub identyfikator podany przez nauczyciela).
- Wymagania wspólne z punktu 1 obowiązują w każdym zadaniu, nawet jeśli treść zadania ich nie powtarza.
- Jeśli w zadaniu podano dane testowe, sprawdź na nich swój program – wynik musi się zgadzać.
- Na egzaminie na jedną aplikację masz w praktyce około 45–60 minut. Mierz czas swojej pracy.

1. Wymagania wspólne dla wszystkich zadań

Poniższe wymagania obowiązują w każdym zadaniu, nawet jeśli treść zadania ich nie powtarza – tak jak w arkuszu egzaminacyjnym.

1.1. Aplikacje desktopowe (WPF)

- Aplikacja napisana w języku C# z wykorzystaniem WPF (.NET), interfejs zdefiniowany w XAML, środowisko Visual Studio.
- Tytuł okna zgodny z treścią zadania, zawierający numer zdającego.
- Kontrolki i zmienne mają znaczące nazwy, np. txtEmail, btnZatwierdz, cmbStanowisko, chkCyfry, rbMala, lstHistoria.
- Kod jest czytelnie sformatowany (wcięcia, odstępy), logika obliczeń wydzielona do metod.
- Aplikacja nie może kończyć działania z błędem przy niepoprawnych danych – wszystkie konwersje wykonywane są bezpiecznie (np. TryParse).

1.2. Dokumentacja

Wskazaną w zadaniu metodę należy poprzedzić komentarzem według wzoru:

```
/**************************************************
 nazwa funkcji:        <tu wstaw nazwę>
 opis funkcji:         <tu wstaw opis działania>
 parametry:            <tu wstaw opis parametrów lub „brak”>
 zwracany typ i opis:  <tu wstaw typ i opis lub „brak”>
 autor:                <numer zdającego>
**************************************************/
```

- Zrzuty ekranu wykonuje się w formacie PNG i nazywa zgodnie z treścią zadania (desktop1.png, desktop2.png itd.). Zrzut obejmuje całe okno aplikacji wraz z ewentualnym komunikatem MessageBox.
- Projekt i zrzuty ekranu zapisuje się w folderze nazwanym numerem zdającego, w podfolderze desktop.
- Zalecane ćwiczenie dodatkowe: krótki opis testów – jakie dane wprowadzono i jakiego wyniku oczekiwano.

2. Przegląd zadań

Nr	Zadanie	Poziom	Kluczowe umiejętności
W1	Rejestracja konta	★☆☆	TextBox, PasswordBox, walidacja danych, komunikaty w TextBlock
W2	Generator haseł i dane pracownika	★★☆	ComboBox, CheckBox, klasa Random, MessageBox, budowanie łańcuchów znaków
W3	Kalkulator BMI	★☆☆	parsowanie liczb (double.TryParse), instrukcje warunkowe, zmiana koloru kontrolki z kodu
W4	Szyfr Cezara	★★☆	operacje na znakach (char), pętle, Slider z wiązaniem danych, schowek systemowy
W5	Mieszalnik kolorów RGB	★★☆	Slider, zdarzenie ValueChanged, SolidColorBrush, formatowanie szesnastkowe, ListBox
W6	Rezerwacja stolika	★★☆	DatePicker, RadioButton, ComboBox, walidacja dat, wyrażenia regularne
W7	Przelicznik walut z historią	★☆☆	Dictionary<string, double>, ComboBox, formatowanie liczb, ListBox
W8	Lista zakupów	★☆☆	własna klasa z nadpisaną metodą ToString, ObservableCollection, ListBox, MessageBox z pytaniem
W9	Quiz wiedzy informatycznej	★★☆	RadioButton z GroupName, GroupBox, zmiana Foreground, blokowanie kontrolek
W10	Zamówienie pizzy	★★☆	RadioButton, CheckBox, wspólna obsługa zdarzeń Checked/Unchecked, obliczenia na bieżąco
W11	Stoper z międzyczasami	★★★	DispatcherTimer, Stopwatch, formatowanie TimeSpan, zarządzanie stanem przycisków
W12	Weryfikacja numeru PESEL	★★★	algorytm sumy kontrolnej, operacje na cyfrach, DateTime, metoda zwracająca bool

3. Zadania – aplikacje desktopowe (WPF)

Zadania W1–W12. Przed rozpoczęciem pracy zapoznaj się z wymaganiami wspólnymi (punkt 1).

Zadanie W1. Rejestracja konta

Poziom: ★☆☆ **Umiejętności:** TextBox, PasswordBox, walidacja danych, komunikaty w TextBlock

Przygotuj aplikację desktopową umożliwiającą założenie konta w serwisie internetowym. Aplikacja sprawdza poprawność wprowadzonych danych i informuje użytkownika o błędach.

Wygląd aplikacji

- Tytuł okna: „Rejestracja konta, wykonał: <numer zdającego>”.
- Tło okna w kolorze #E3F2FD, szerokość okna 400 px.
- Nagłówek „Utwórz konto” – czcionka 18 pt, pogrubiona, wyśrodkowany.
- Etykiety i pola: „E-mail” (TextBox), „Hasło” (PasswordBox), „Powtórz hasło” (PasswordBox).
- Przycisk „Zatwierdź” o szerokości 120 px, wyśrodkowany.
- Pod przyciskiem kontrolka TextBlock przeznaczona na komunikaty (domyślnie pusta).

Działanie aplikacji

- Po kliknięciu „Zatwierdź” dane są sprawdzane w podanej kolejności, a wyświetlany jest tylko pierwszy napotkany błąd:
  - jeśli adres e-mail nie zawiera znaku „@” – komunikat „Nieprawidłowy adres e-mail”,
  - jeśli hasło ma mniej niż 6 znaków – komunikat „Hasło musi mieć co najmniej 6 znaków”,
  - jeśli hasła w obu polach są różne – komunikat „Hasła się różnią”.
- Komunikaty o błędach wyświetlane są w kolorze czerwonym.
- Jeśli dane są poprawne, w TextBlock pojawia się w kolorze zielonym tekst „Witaj <adres e-mail>”, a oba pola haseł są czyszczone.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – błędny adres e-mail
- desktop2.png – różne hasła
- desktop3.png – poprawna rejestracja

Opatrz metodę obsługi przycisku komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W2. Generator haseł i dane pracownika

Poziom: ★★☆ **Umiejętności:** ComboBox, CheckBox, klasa Random, MessageBox, budowanie łańcuchów znaków

Przygotuj aplikację wspomagającą dział IT przy tworzeniu kont nowych pracowników. Aplikacja zbiera dane pracownika i generuje dla niego hasło o zadanych parametrach.

Wygląd aplikacji

- Tytuł okna: „Nowy pracownik, wykonał: <numer zdającego>”.
- Okno podzielone na dwie kolumny (Grid), każda umieszczona w kontrolce GroupBox, tło okna #E0F2F1.
- Lewa kolumna „Dane pracownika”: pola „Imię”, „Nazwisko” (TextBox) oraz „Stanowisko” (ComboBox z pozycjami: Kierownik, Starszy programista, Młodszy programista, Tester; domyślnie wybrany Kierownik).
- Prawa kolumna „Generowanie hasła”: pole „Ile znaków?” (TextBox), pola wyboru: „Małe i wielkie litery” (domyślnie zaznaczone), „Cyfry”, „Znaki specjalne” oraz przycisk „Generuj hasło”.
- Pod kolumnami, wyśrodkowany, przycisk „Zatwierdź” o tle #00695C i białym tekście.

Działanie aplikacji

- Po kliknięciu „Generuj hasło” sprawdzane jest, czy w polu „Ile znaków?” wpisano liczbę całkowitą z zakresu 4–30. W przeciwnym razie MessageBox z komunikatem „Podaj liczbę znaków od 4 do 30”.
- Hasło zawsze może zawierać małe litery. Zaznaczenie pól dopuszcza dodatkowo: wielkie litery, cyfry, znaki specjalne ze zbioru !@#$%^&*()_+-=.
- Wygenerowane hasło wyświetlane jest w oknie MessageBox i zapamiętywane w zmiennej.
- Po kliknięciu „Zatwierdź” wyświetlany jest MessageBox o treści: „Dane pracownika: <imię> <nazwisko> <stanowisko> Hasło: <hasło>”.
- Jeśli hasło nie zostało jeszcze wygenerowane, zamiast danych wyświetlany jest komunikat „Najpierw wygeneruj hasło”.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – okno z wypełnionymi danymi
- desktop2.png – MessageBox z hasłem
- desktop3.png – MessageBox z danymi pracownika

Opatrz metodę generującą hasło komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W3. Kalkulator BMI

Poziom: ★☆☆ **Umiejętności:** parsowanie liczb (double.TryParse), instrukcje warunkowe, zmiana koloru kontrolki z kodu

Przygotuj aplikację obliczającą wskaźnik masy ciała BMI i wskazującą kategorię wyniku.

Wygląd aplikacji

- Tytuł okna: „Kalkulator BMI, wykonał: <numer zdającego>”.
- Pola: „Masa ciała [kg]” oraz „Wzrost [cm]” (TextBox), przycisk „Oblicz”.
- Kontrolka TextBlock na wynik, czcionka 16 pt.
- Obok wyniku kwadrat (Rectangle) 40×40 px, domyślnie w kolorze szarym – wskaźnik kategorii.

Działanie aplikacji

- BMI = masa / (wzrost w metrach)², wynik zaokrąglony do dwóch miejsc po przecinku.
- Aplikacja akceptuje zarówno przecinek, jak i kropkę jako separator dziesiętny.
- Kategorie i kolory kwadratu: poniżej 18,5 – „Niedowaga” (niebieski); 18,5–24,99 – „Waga prawidłowa” (zielony); 25–29,99 – „Nadwaga” (pomarańczowy); 30 i więcej – „Otyłość” (czerwony).
- Wynik prezentowany w formie: „Twoje BMI: 22,86 – Waga prawidłowa”.
- Walidacja: masa musi być liczbą z zakresu 20–300, wzrost 50–250. Przy błędzie – MessageBox „Wprowadź poprawne dane” z ikoną błędu.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – wynik w kategorii „Waga prawidłowa”
- desktop2.png – wynik w kategorii „Otyłość”
- desktop3.png – komunikat o błędnych danych

Opatrz metodę obliczającą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W4. Szyfr Cezara

Poziom: ★★☆ **Umiejętności:** operacje na znakach (char), pętle, Slider z wiązaniem danych, schowek systemowy

Przygotuj aplikację szyfrującą i odszyfrowującą tekst szyfrem Cezara.

Wygląd aplikacji

- Tytuł okna: „Szyfr Cezara, wykonał: <numer zdającego>”.
- Wielowierszowe pole tekstowe „Tekst” (AcceptsReturn, wysokość 100 px).
- Suwak „Klucz” (Slider) w zakresie 1–25, domyślnie 3, z krokiem całkowitym; obok TextBlock wyświetlający aktualną wartość suwaka (wiązanie danych – Binding).
- Przyciski „Szyfruj” i „Odszyfruj” umieszczone obok siebie.
- Pole „Wynik” tylko do odczytu (IsReadOnly) oraz przycisk „Kopiuj do schowka”.

Działanie aplikacji

- Szyfrowanie przesuwa każdą literę alfabetu łacińskiego (a–z, A–Z) o wartość klucza, z zawinięciem (z → c przy kluczu 3). Wielkość liter jest zachowana.
- Pozostałe znaki (cyfry, spacje, polskie znaki, interpunkcja) pozostają bez zmian.
- Odszyfrowanie przesuwa litery w przeciwnym kierunku.
- Szyfrowanie realizuje osobna metoda przyjmująca tekst i klucz oraz zwracająca zaszyfrowany tekst; odszyfrowanie korzysta z tej samej metody.
- „Kopiuj do schowka” kopiuje wynik (Clipboard.SetText) i wyświetla MessageBox „Skopiowano”.
- Test kontrolny: tekst „Ala ma kota!” z kluczem 3 daje „Dod pd nrwd!”.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – zaszyfrowany tekst
- desktop2.png – odszyfrowany tekst

Opatrz metodę szyfrującą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W5. Mieszalnik kolorów RGB

Poziom: ★★☆ **Umiejętności:** Slider, zdarzenie ValueChanged, SolidColorBrush, formatowanie szesnastkowe, ListBox

Przygotuj aplikację pozwalającą grafikowi dobrać kolor ze składowych RGB i zapisać ulubione kolory.

Wygląd aplikacji

- Tytuł okna: „Mieszalnik kolorów, wykonał: <numer zdającego>”.
- Trzy suwaki (Slider) o zakresie 0–255 z etykietami „R”, „G”, „B”; przy każdym TextBlock z bieżącą wartością.
- Prostokąt podglądu 200×100 px z obramowaniem 1 px.
- TextBlock z kodem koloru, czcionka Consolas 20 pt.
- Przycisk „Zapisz kolor”, lista ListBox z zapisanymi kolorami oraz przycisk „Usuń zaznaczony”.

Działanie aplikacji

- Każda zmiana suwaka natychmiast aktualizuje wartość liczbową, kolor prostokąta oraz kod koloru w formacie #RRGGBB (wielkie litery, np. #FF8000).
- „Zapisz kolor” dodaje kod do listy. Jeśli kolor już jest na liście – MessageBox „Ten kolor jest już zapisany”.
- Zaznaczenie pozycji na liście ustawia suwaki na wartości tego koloru.
- „Usuń zaznaczony” usuwa zaznaczony kolor; jeśli nic nie zaznaczono – komunikat „Zaznacz kolor do usunięcia”.
- Rozszerzenie (opcjonalnie): tło każdej pozycji listy w kolorze, który opisuje.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – dobrany kolor i lista co najmniej trzech kolorów
- desktop2.png – komunikat o duplikacie

Opatrz metodę aktualizującą kolor komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W6. Rezerwacja stolika

Poziom: ★★☆ **Umiejętności:** DatePicker, RadioButton, ComboBox, walidacja dat, wyrażenia regularne

Przygotuj aplikację dla restauracji umożliwiającą przyjęcie rezerwacji stolika.

Wygląd aplikacji

- Tytuł okna: „Rezerwacja stolika, wykonał: <numer zdającego>”.
- Pola: „Imię i nazwisko”, „Telefon” (TextBox), „Data” (DatePicker), „Godzina” (ComboBox z godzinami 12:00–21:00 co godzinę, wypełniany w kodzie), „Liczba osób” (ComboBox 1–8).
- Pole wyboru „Stolik przy oknie” oraz dwa przyciski opcji „Sala główna” (domyślny) i „Ogródek”.
- Przycisk „Zarezerwuj” o tle #6D4C41 i białym tekście.
- Pod przyciskiem obramowanie (Border) z TextBlock na podsumowanie rezerwacji.

Działanie aplikacji

- Wszystkie pola muszą być wypełnione – w przeciwnym razie MessageBox „Uzupełnij wszystkie pola”.
- Telefon musi składać się z dokładnie 9 cyfr (Regex).
- Data nie może być wcześniejsza niż dzisiejsza ani późniejsza niż 30 dni od dziś.
- Ogródek jest dostępny tylko w miesiącach maj–wrzesień; w innym przypadku komunikat „Ogródek jest nieczynny w wybranym terminie”.
- Po poprawnej walidacji w ramce pojawia się podsumowanie, np.: „Rezerwacja: Jan Kowalski, 12.05.2027, 18:00, 4 osoby, ogródek, przy oknie”.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – poprawna rezerwacja
- desktop2.png – błąd daty
- desktop3.png – ogródek poza sezonem

Opatrz metodę walidującą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W7. Przelicznik walut z historią

Poziom: ★☆☆ **Umiejętności:** Dictionary<string, double>, ComboBox, formatowanie liczb, ListBox

Przygotuj aplikację kantoru przeliczającą kwoty między walutami według stałych kursów.

Wygląd aplikacji

- Tytuł okna: „Kantor, wykonał: <numer zdającego>”.
- Pole „Kwota” (TextBox), dwie listy rozwijane „Z waluty” i „Na walutę” (PLN, EUR, USD, GBP, CHF).
- Między listami przycisk „⇄” zamieniający waluty miejscami.
- Przycisk „Przelicz” oraz TextBlock z wynikiem, czcionka 16 pt, pogrubiona.
- Pod spodem ListBox „Historia” z nagłówkiem.

Działanie aplikacji

- Kursy w złotych przechowywane w słowniku: EUR = 4,30; USD = 3,95; GBP = 5,05; CHF = 4,45; PLN = 1.
- Przeliczenie: kwota × kurs waluty źródłowej / kurs waluty docelowej, wynik z dwoma miejscami po przecinku.
- Wynik w formie: „100,00 EUR = 430,00 PLN”.
- Kwota musi być liczbą większą od zera; ta sama waluta po obu stronach – komunikat „Wybierz różne waluty”.
- Każde przeliczenie trafia na początek historii; historia przechowuje maksymalnie 5 ostatnich pozycji.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – wynik i historia z co najmniej trzema pozycjami
- desktop2.png – komunikat o błędnej kwocie

Opatrz metodę przeliczającą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W8. Lista zakupów

Poziom: ★☆☆ **Umiejętności:** własna klasa z nadpisaną metodą ToString, ObservableCollection, ListBox, MessageBox z pytaniem

Przygotuj aplikację do tworzenia listy zakupów. Pozycje listy są obiektami klasy Produkt.

Wygląd aplikacji

- Tytuł okna: „Lista zakupów, wykonał: <numer zdającego>”.
- W jednym wierszu: pole „Produkt” (TextBox), pole „Ilość” (TextBox, szerokość 60 px), lista „Jednostka” (szt., kg, l, opak.) i przycisk „Dodaj”.
- Pod spodem ListBox o wysokości 200 px.
- Przyciski „Usuń zaznaczony” i „Wyczyść listę”.
- Na dole TextBlock „Pozycji na liście: N”.

Działanie aplikacji

- Klasa Produkt ma pola: Nazwa, Ilosc (double), Jednostka oraz nadpisaną metodę ToString zwracającą np. „Mleko – 2 l”.
- Lista wiązana z kolekcją ObservableCollection<Produkt>.
- Walidacja: nazwa niepusta, ilość większa od 0.
- Jeśli produkt o tej samej nazwie i jednostce jest już na liście (bez względu na wielkość liter), zwiększana jest jego ilość zamiast dodawania nowej pozycji.
- „Wyczyść listę” wyświetla pytanie „Czy na pewno usunąć wszystkie pozycje?” (przyciski Tak/Nie).
- Licznik pozycji aktualizowany po każdej operacji.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – lista z co najmniej czterema produktami
- desktop2.png – pytanie przed wyczyszczeniem listy

Opatrz metodę dodającą produkt komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W9. Quiz wiedzy informatycznej

Poziom: ★★☆ **Umiejętności:** RadioButton z GroupName, GroupBox, zmiana Foreground, blokowanie kontrolek

Przygotuj aplikację z krótkim testem wiedzy z zakresu kwalifikacji INF.04.

Wygląd aplikacji

- Tytuł okna: „Quiz, wykonał: <numer zdającego>”.
- Trzy pytania, każde w osobnym GroupBox, z trzema przyciskami opcji (każde pytanie ma własną wartość GroupName).
- Pytanie 1: „Który typ w C# służy do przechowywania liczb całkowitych?” – string / int (poprawna) / bool.
- Pytanie 2: „Które polecenie SQL usuwa rekordy z tabeli?” – DROP / DELETE (poprawna) / REMOVE.
- Pytanie 3: „Jak nazywa się hook React do przechowywania stanu?” – useEffect / useRef / useState (poprawna).
- Przyciski „Sprawdź” i „Od nowa”, TextBlock na wynik (czcionka 16 pt).

Działanie aplikacji

- Jeśli nie udzielono odpowiedzi na wszystkie pytania – MessageBox „Odpowiedz na wszystkie pytania”.
- Po sprawdzeniu poprawna odpowiedź w każdym pytaniu ma zielony kolor tekstu, a błędna zaznaczona przez użytkownika – czerwony.
- Wynik w formie: „Twój wynik: 2/3 (67%)”.
- Po sprawdzeniu wszystkie przyciski opcji są zablokowane (IsEnabled = false).
- „Od nowa” odblokowuje i odznacza przyciski, przywraca kolor czarny i czyści wynik.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – wynik z co najmniej jedną błędną odpowiedzią
- desktop2.png – komunikat o brakujących odpowiedziach

Opatrz metodę sprawdzającą komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W10. Zamówienie pizzy

Poziom: ★★☆ **Umiejętności:** RadioButton, CheckBox, wspólna obsługa zdarzeń Checked/Unchecked, obliczenia na bieżąco

Przygotuj aplikację pizzerii, w której klient konfiguruje pizzę, a cena jest przeliczana natychmiast po każdej zmianie.

Wygląd aplikacji

- Tytuł okna: „Pizzeria, wykonał: <numer zdającego>”.
- Sekcja „Rozmiar”: Mała – 25 zł, Średnia – 32 zł (domyślna), Duża – 40 zł.
- Sekcja „Dodatki”: ser (4 zł), szynka (5 zł), pieczarki (3 zł), oliwki (4 zł), jalapeño (3 zł).
- Lista „Sos”: pomidorowy (0 zł), czosnkowy (2 zł), BBQ (2 zł).
- Pole wyboru „Dostawa (+8 zł, gratis od 60 zł)”.
- TextBlock „Do zapłaty: 32,00 zł” – czcionka 20 pt, kolor #C62828; przycisk „Zamów”.

Działanie aplikacji

- Cena przeliczana natychmiast po każdej zmianie rozmiaru, dodatku, sosu lub dostawy (jedna wspólna metoda wywoływana przez wszystkie zdarzenia).
- Dostawa kosztuje 8 zł, chyba że wartość pizzy (bez dostawy) wynosi co najmniej 60 zł.
- Można wybrać maksymalnie 4 dodatki – po zaznaczeniu czwartego pozostałe niezaznaczone pola są blokowane; odznaczenie przywraca możliwość wyboru.
- „Zamów” wyświetla MessageBox z podsumowaniem: rozmiar, lista dodatków (lub „brak”), sos, dostawa, kwota.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – zamówienie z 4 dodatkami (widoczne zablokowane pole)
- desktop2.png – podsumowanie zamówienia

Opatrz metodę obliczającą cenę komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W11. Stoper z międzyczasami

Poziom: ★★★ **Umiejętności:** DispatcherTimer, Stopwatch, formatowanie TimeSpan, zarządzanie stanem przycisków

Przygotuj aplikację stopera sportowego z możliwością zapisywania międzyczasów.

Wygląd aplikacji

- Tytuł okna: „Stoper, wykonał: <numer zdającego>”.
- TextBlock z czasem w formacie „mm:ss.d” (np. „00:00.0”) – czcionka Consolas 48 pt, wyśrodkowany.
- Przyciski: „Start” (tło zielone), „Międzyczas”, „Reset”.
- ListBox z międzyczasami oraz TextBlock „Najszybsze okrążenie: –”.

Działanie aplikacji

- Czas mierzony klasą Stopwatch, wyświetlanie odświeżane przez DispatcherTimer co 100 ms.
- Przycisk „Start” po uruchomieniu zmienia treść na „Stop” i kolor tła na czerwony; ponowne kliknięcie zatrzymuje pomiar i przywraca „Start”.
- „Międzyczas” jest aktywny tylko podczas pomiaru i dodaje pozycję „Okr. N – mm:ss.d”, gdzie czas to długość danego okrążenia (od poprzedniego międzyczasu).
- „Reset” jest aktywny tylko po zatrzymaniu stopera; zeruje czas, listę i informację o najszybszym okrążeniu.
- Po każdym międzyczasie aktualizowana jest informacja o najszybszym okrążeniu, np. „Najszybsze okrążenie: Okr. 2 – 00:11.8”.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – działający stoper z co najmniej trzema międzyczasami
- desktop2.png – stoper zatrzymany

Opatrz metodę obsługi międzyczasu komentarzem zgodnym ze wzorem z punktu 1.2.

Zadanie W12. Weryfikacja numeru PESEL

Poziom: ★★★ **Umiejętności:** algorytm sumy kontrolnej, operacje na cyfrach, DateTime, metoda zwracająca bool

Przygotuj aplikację sprawdzającą poprawność numeru PESEL i odczytującą z niego datę urodzenia oraz płeć.

Wygląd aplikacji

- Tytuł okna: „Weryfikacja PESEL, wykonał: <numer zdającego>”.
- Pole „Numer PESEL” (TextBox, MaxLength = 11) i przycisk „Sprawdź”.
- Ramka (Border) z trzema wierszami: „Poprawność:”, „Data urodzenia:”, „Płeć:”.
- Obramowanie ramki domyślnie szare, grubość 2 px.

Działanie aplikacji

- PESEL musi składać się z 11 cyfr – w przeciwnym razie komunikat „PESEL musi zawierać 11 cyfr”.
- Suma kontrolna: pierwsze 10 cyfr mnożone kolejno przez wagi 1, 3, 7, 9, 1, 3, 7, 9, 1, 3; cyfra kontrolna = (10 − suma mod 10) mod 10 i musi być równa cyfrze 11.
- Stulecie z miesiąca: 01–12 → 1900–1999; 21–32 → 2000–2099 (miesiąc − 20); 81–92 → 1800–1899 (miesiąc − 80). Data musi istnieć w kalendarzu.
- Płeć: dziesiąta cyfra parzysta – kobieta, nieparzysta – mężczyzna.
- Sprawdzenie realizuje metoda bool CzyPoprawnyPesel(string pesel). Przy poprawnym numerze ramka ma kolor zielony i wyświetlane są dane, przy błędnym – kolor czerwony, a pola danych zawierają „–”.
- Dane testowe: 44051401359 – poprawny (14.05.1944, mężczyzna); 02270812343 – poprawny (08.07.2002, kobieta); 44051401358 – niepoprawny.

Dokumentacja

Wykonaj zrzuty ekranu:

- desktop1.png – poprawny PESEL z XX wieku
- desktop2.png – poprawny PESEL z XXI wieku
- desktop3.png – niepoprawny PESEL

Opatrz metodę CzyPoprawnyPesel komentarzem zgodnym ze wzorem z punktu 1.2.